

load balancer inbetween would be needed. At that point, you could be faced with a new problem – managing user sessions. Normally, PHP manages sessions automatically by using files. These files are not automatically available to other servers with the same app. To resolve this problem, PHP allows you to use a database for storing (reading and writing) user sessions, so that the sessions could be read and written from any of the multiple servers running the same PHP code. Any one of the many guides on the Internet can help you gain an insight into this, but the fact is that in order to use the feature, you just need to set the values of `DB_BACKED_SESSION_HANDLER_DB_INDEX` and `DB_BACKED_SESSION_HANDLER_TABLE_NAME` correctly in the `configuration.inc.php` file. QCubed will take care of the rest. The importance and meaning of those variables is explained in the file.

Centralised FormState handling

QCubed saves the FormState (state of the application) on the server side to help make coding easier. However, saving the FormState is itself messy! The problem is, to save FormStates, you need a location! There are a few options provided by QCubed, as listed below.

1. **QFormStateHandler:** Saves the FormState as a hidden variable within the page itself. The advantage is that it does not consume any space on the server side. The disadvantage is that it makes the pages huge and bulky. One should use it only when nothing else works!
2. **QSessionFormStateHandler:** Saves the FormState as part of the user session. The downside here is that, with time, it uses a lot of session data and can slow down the user session or even crash the session. However, it makes more sense because all FormStates are cleared when the user logs out and the session is destroyed. This option is the default.
3. **QFileFormStateHandler:** This saves the FormStates as individual files on the server side; this is much better than `QSessionFormStateHandler` because it does not make the session swell. The disadvantage of this (and `QSessionFormStateHandler`) is that if there are more than one servers running the same application, no server knows about the FormStates created by the others – a curse for a distributed app!
4. **QDbBackedFormStateHandler:** Create a table with the needed structure (explained in *configuration.inc.php*) in one of your databases, and set the correct values in the configuration file. QCubed will start maintaining the FormStates in the assigned table! Interestingly, if the database-based session handler and `QSessionFormStateHandler` are used, the FormStates get saved in the database (as part of session data) as well. However, `QDbBackedFormStateHandler` will operate faster. This is the FormState handler that should be used if a considerable amount of traffic is expected or received and it is required to distribute the application load on more than one server.

All this helps your application scale as and when needed, without requiring you to think about scaling at the very beginning.

Security

We have already mentioned QQuery – QCubed's way of querying the databases—and that it automatically escapes the strings before sending them to the database so that it does not have to be done manually. Since queries are not written by hand, it results in better security of the application and the chances of missing 'that one spot' are almost nil.

However, security woes are not limited to SQL injection attempts by malicious users. One of the attack techniques called Cross Site Scripting (XSS) does not bother the website itself but can be used to steal data from machines of other users visiting the site or app. As developers, it becomes our responsibility to protect our users from attacks by others.

XSS attacks are carried out by inputting valid but dangerous HTML code in those input boxes of a website that will cause the input to be displayed somewhere, e.g., a comment input box would cause the input entered into it to be displayed on at least one page. If dangerous JavaScript is entered into such input boxes, it can result in an output that might seem innocent and harmless, but can extract data from the user's browser. With the advent of HTML5's more powerful JS API, it can wreak havoc on the user's computer.

To prevent this, QCubed comes with built-in XSS prevention. All that needs to be done is to set the 'CrossScripting' property of the concerned textbox to `QCrossScripting::Deny` or `QCrossScripting::HTMLPurifier`. The former avoids any dangerous HTML from being entered and raises an exception, while the latter utilises the `HTMLPurifier` library to filter out the dangerous HTML and purify it. While the defaults set by the QCubed framework are good enough, you can learn more about how to use them in projects, in detail, at <http://goo.gl/F33C>.

Recent developments

By the time you read this, QCubed 2.2.0 would be out with new improvements:

- A friendly installer to assist in installing the framework.
- An improved QQuery with 'having' clause support.
- An upgraded JQuery UI version.
- QCubed code moved to GitHub at <https://github.com/qcubed/>.
- QCubed API documentation at <http://qcdocs.c-integration.com/>. This is currently a work-in-progress and may not be perfect at the moment, but do bear with it for some time. 

By: Vaibhav Kaushal

The author is a core contributor to QCubed. He gives QCubed related advice on his website www.c-integration.com.